Анализ и концептуальное моделирование систем

профессор, д.т.н. Смоленцева Т.Е.:
smolenceva@mirea.ru:

ЛЕКЦИЯ 6.
Модели анализа, проектирования, реализации.

СОДЕРЖАНИЕ ЛЕКЦИИ:

- Модель реализации.
- Правила и рекомендации по построению диаграмм.

1. МОДЕЛЬ РЕАЛИЗАЦИИ

Основная цель, преследуемая при построении данной модели – получение работоспособной версии системы.

Помимо непосредственного написания программного кода будущей системы, на данной стадии окончательно определяется логическая и физическая организация классов в виде компонентов и подсистем, а также топология распределенной информационной системы.

При разработке модели преследуются следующие цели:

- определение окончательного состава, структуры и кода классов;
- распределение классов по компонентам и подсистемам;
- определение топологии распределенной системы и распределение подсистем по узлам сети;
- планирование итераций (версий) сборки системы;
- сборка версий системы.

При разработке модели реализации рекомендуется построить диаграммы (основные артефакты):

- **компонентов;**
- **развертывания.**

К основным артефактам также относятся исходные тексты программ, план реализации версий системы и частично или полностью работоспособные версии системы.

При разработке диаграмм компонентов преследуются следующие цели:

- спецификация общей структуры исходного кода системы;
- спецификация исполнимого варианта системы.

Данная диаграмма обеспечивает согласованный переход от логического к физическому представлению системы в виде программных компонентов.

Одни компоненты могут существовать только на этапе компиляции программного кода, другие – на этапе его исполнения.

Основными элементами диаграммы являются:

- **компоненты;**
- **интерфейсы и зависимости между ними.**

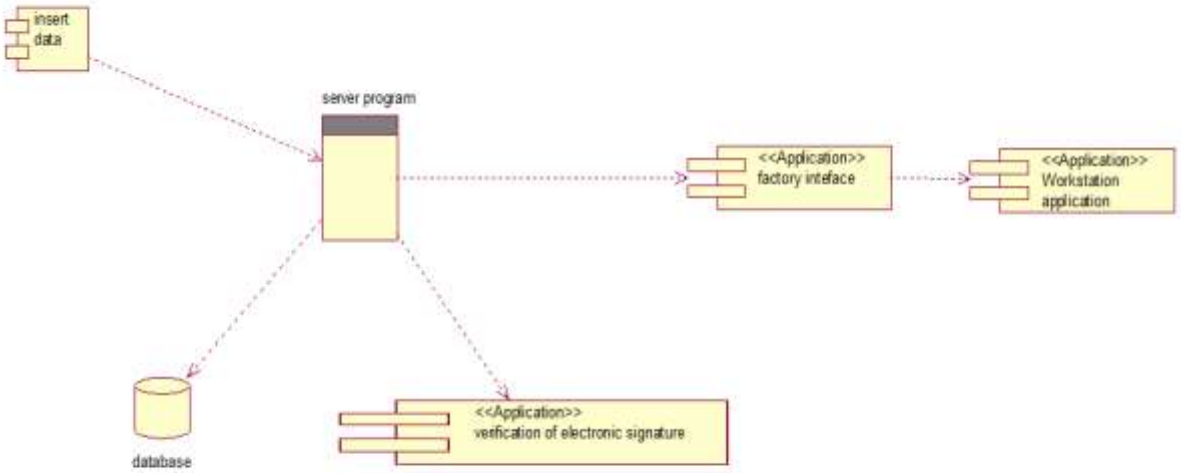
Кроме этого на ней могут отображаться ключевые классы, входящие в компоненты.

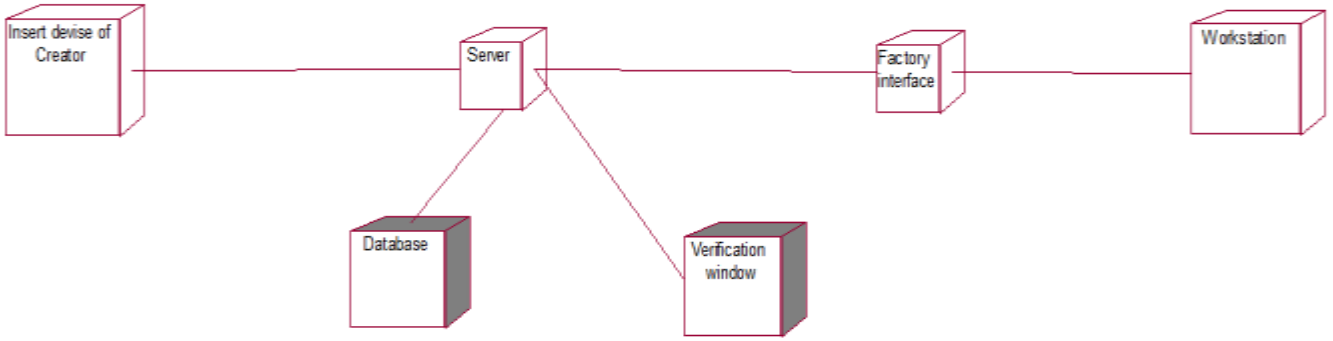
Стандартные стереотипы:

- **«file»** - любой файл, кроме таблицы;
- **«executable»** - программа (исполняемый файл);
- **«library»** - статическая или динамическая библиотека;
- **«source»** - файл с исходным текстом программы;
- **«document»** - остальные файлы (например, файл справки);
- **«table»** - таблица базы данных.

Стереотип	Условное обозначение
executable	
library	
table	
file	
document	

Интерфейс (interface, англ.) – это внешне видимый, именованный набор операций, который класс, компонент или подсистема может предоставить другому классу, компоненту или подсистеме, для выполнения им своих функций.

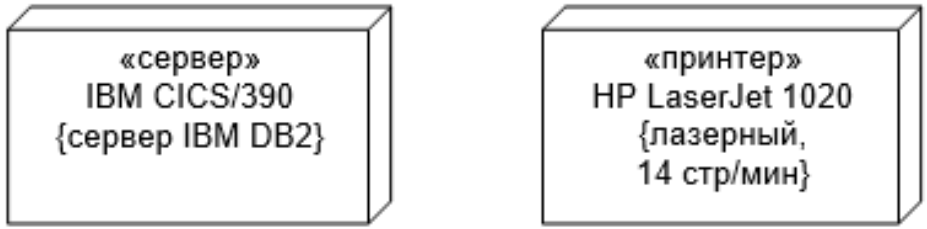

insert data
server program
<<Application>>
factory inteface
<<Application>>
Workstation application
database
<<Application>>
verification of electronic signature


Insert devise of Creator
Server
Factory interface
Workstation
Database
Verification window

Второй формой физического представления программной системы является **диаграмма размещения (развертывания)**.

Основные цели, преследуемые при разработке диаграммы развертывания:

- распределение компонентов системы по ее физическим узлам;
- отображение физических связей между узлами системы на этапе исполнения;
- выявление узких мест системы и реконфигурация для достижения требуемой производительности.



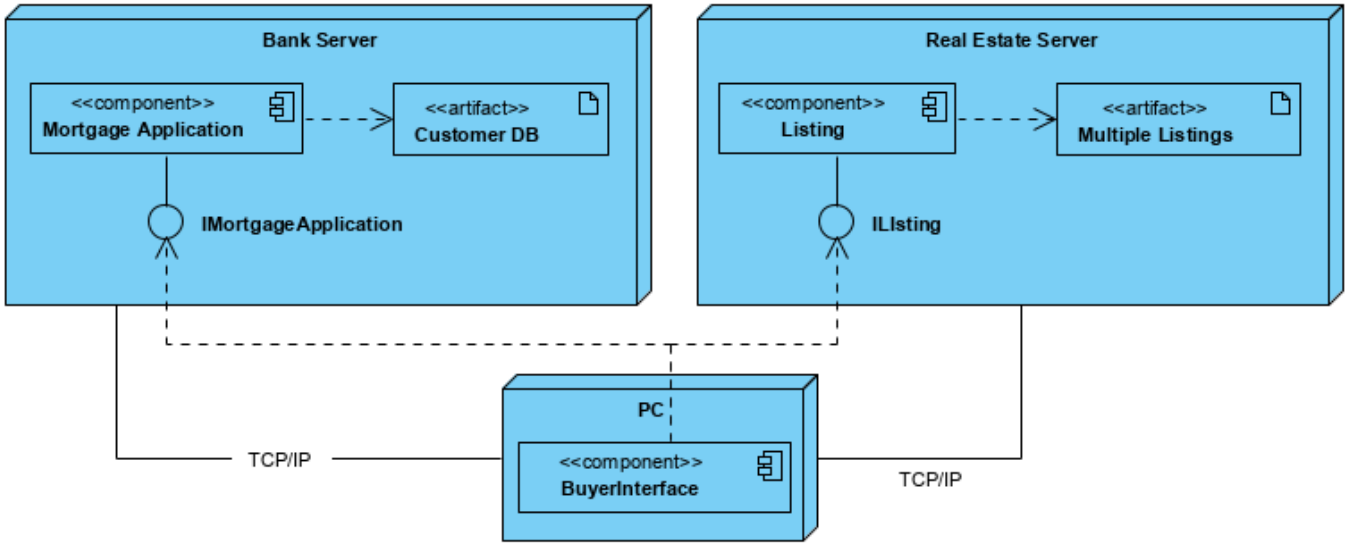
Элементами диаграммы реализации являются узлы, компоненты и связи между ними.

Узел (node, англ.) *представляет собой некоторый физически существующий элемент системы. В качестве узла могут рассматриваться компьютеры, датчики, принтеры, модемы, цифровые камеры, сканеры и т.д.*
В качестве узла можно отображать его экземпляр. В этом случае перед именем узла ставиться двоеточие и оно подчеркивается.

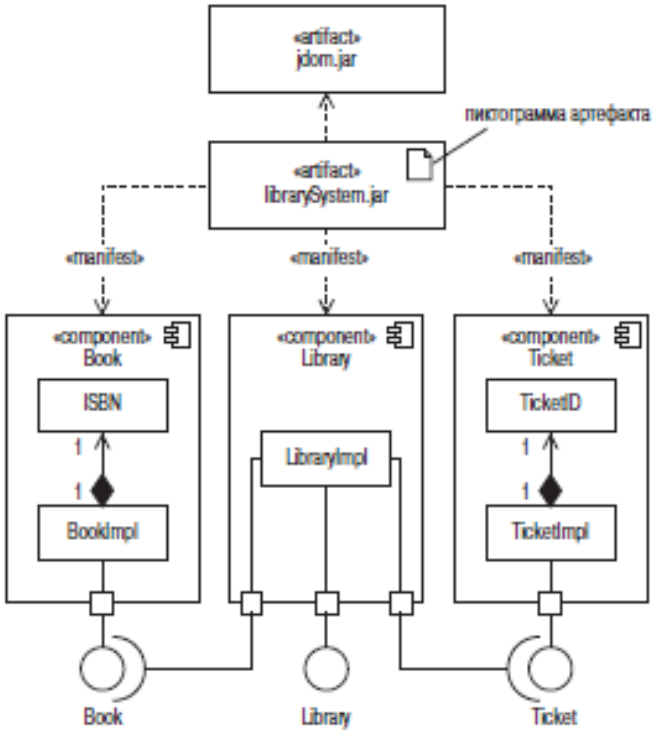
Существует два стандартных стереотипа для узлов:

- *«device» (устройство) – узел представляет тип физического устройства, например ПК или сервер Fire корпорации Sun.*
- *«execution environment» (среда выполнения) – узел представляет тип среды выполнения программного обеспечения, например вебсервер Apache или EJBконтейнер (Enterprise JavaBeans) JBoss.*

Экземпляр узла представляет конкретный вычислительный ресурс.



Для стереотипов можно придумать собственные пиктограммы, напоминающие реальное оборудование, и затем использовать эти символы на диаграмме развертывания. Такой подход упрощает восприятие диаграммы.



Артефакт представляет описание реальной сущности, например, такой как файл.

Экземпляр артефакта представляет конкретный экземпляр конкретного артефакта.

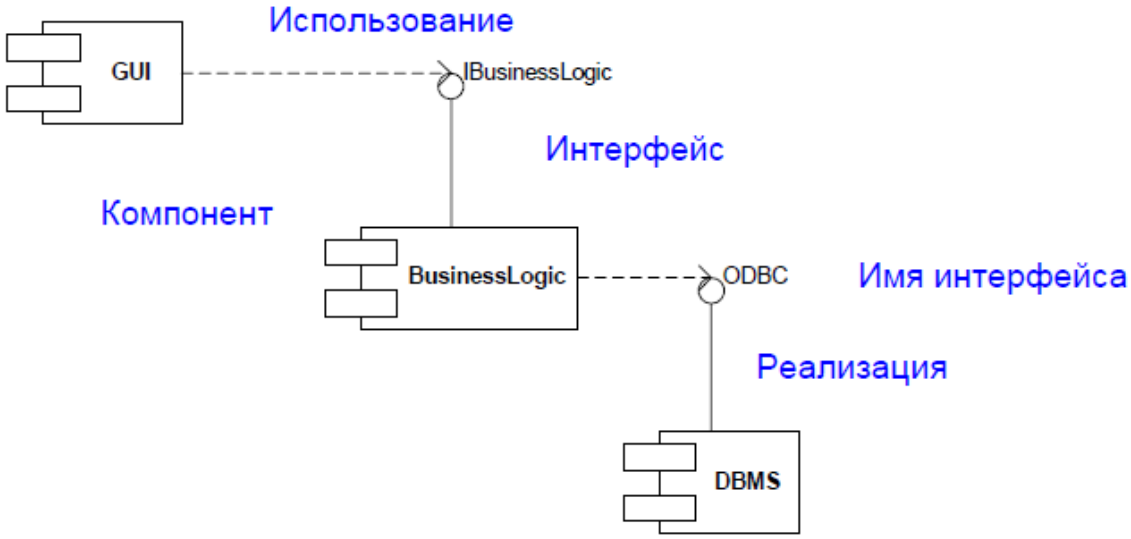
Артефакты могут представлять один или более компонентов.

На диаграмме компонентов применяются следующие основные типы сущностей:

- *компоненты;*
- *интерфейсы;*
- *классы;*
- *объекты.*

На диаграмме компонентов обычно используются отношения следующих типов:

- *зависимость;*
- *ассоциация (главным образом в форме композиции);*
- *реализация.*



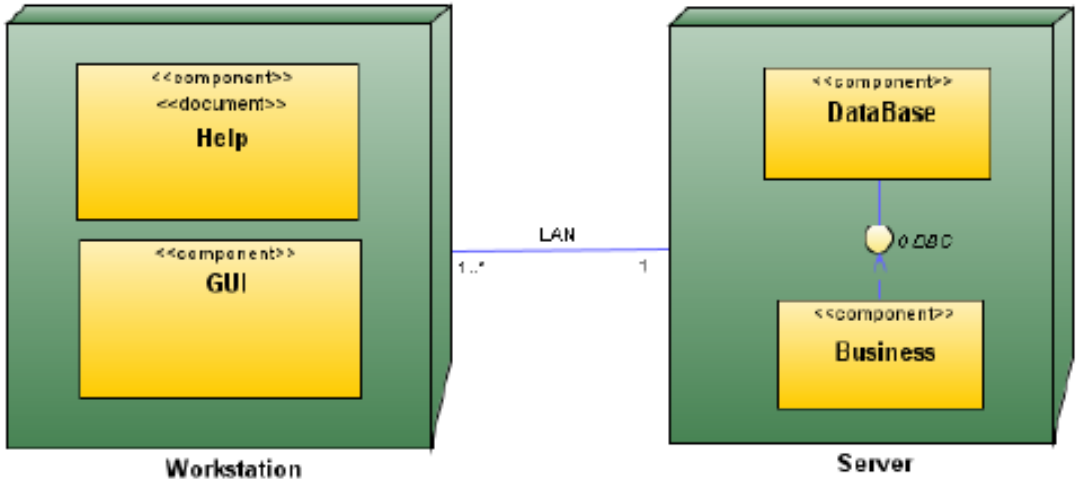
Если приложение поставляется в виде "конструктора" (набора "кубиков" — компонентов) из которого при установке собирается конкретный уникальный экземпляр приложения, то диаграммы компонентов оказываются незаменимым средством.

При разработке приложений, которые должны взаимодействовать с так называемыми унаследованными приложениями и данными, без диаграмм компонентов также трудно обойтись.

Фактически единственным средством UML, позволяющим как-то описать и включить в модель унаследованные приложения и данные являются компоненты (и их интерфейсы). Сюда же относится случай моделирования физической структуры данных в базе, работать с которой требуется помимо "родной" СУБД.

Примером применения диаграмм компонентов является моделирование систем динамической архитектуры, то есть таких систем, которые меняют состав и количество своих компонентов во время выполнения.

На диаграмме размещения, по сравнению с диаграммами компонентов, применяется только один дополнительный тип сущности — узел и два дополнительных отношения: ассоциация между узлами и размещение компонента на узле. В остальном диаграммы размещения наследуют возможности диаграмм компонентов.



Структура сложной системы описывается на уровне дескрипторов.

Диаграммы классов моделируют структуру объектов и связей между ними.

Классы выбираются на основе анализа предметной области, взаимного согласования элементов модели и общих теоретических соображений.

Диаграмма объектов — это пример связей программных объектов в отдельный момент выполнения системы.

Диаграммы компонентов моделируют структуру компонентов (артефактов) и взаимосвязей между ними.

Диаграммы размещения моделируют структуру вычислительных ресурсов и размещение компонентов.

2. ПРАВИЛА И РЕКОМЕНДАЦИИ ПО ПОСТРОЕНИЮ ДИАГРАММ

При разработке диаграммы классов следует придерживаться следующих правил и рекомендаций.

1. За основу диаграммы классов при ее разработке берется диаграмма классов анализа.
2. Для классов должны быть определены и специфицированы все атрибуты и методы. Их спецификация, как правило, выполняется с учетом выбранного языка программирования.
3. При определении методов рекомендуется использовать сообщения с ранее разработанных диаграмм последовательности и коммуникации.
4. Большинство современных средств разработки поддерживает визуальную разработку интерфейса системы – меню, диалоговых форм, элементов диалоговых окон, панелей инструментов и т. д. В качестве исходных данных для их проектирования служат прототипы пользовательских интерфейсов. В связи с этим при проектировании таких классов основное внимание следует уделять особенностям отображения информации и специфичным операциям, которые возникают при диалоге пользователя с системой. Граничные классы, определяющие интерфейс взаимодействия с другими системами, требуют детального проектирования.
5. Для проектирования классов-сущностей можно применять подходы, используемые при проектировании БД.
6. Несмотря на то, что каждому объекту при выполнении программы автоматически назначается уникальный идентификатор, рекомендуется для классов-сущностей явно определять атрибуты, хранящие значения первичного ключа.

7. В отличие от реляционных БД поощряется использование в классах многозначных атрибутов в виде массивов, множеств, списков и т. д.
8. Управляющие классы следует проектировать только в случаях крайней необходимости – управления сложным взаимодействием объектов, реализации сложной бизнес-логики и вычислений, контроля целостности объектов и т. п. В противном случае функциональность этого класса лучше распределить между соответствующими граничными классами и классами-сущностями.
9. Для атрибутов рекомендуется назначать видимость private (закрытый) или protected (защищенный). Если требуется чтение значения такого атрибута из объектов других классов, то следует предусмотреть для него get-метод, а если возможность установки нового значения – set-метод.
10. Для методов видимость public (общедоступный) следует устанавливать только в случае крайней необходимости.
11. Ввиду большого количества классов в системе рекомендуется диаграммы классов разрабатывать отдельно для каждого пакета.

При разработке диаграммы деятельности следует придерживаться следующих правил:

1. При построении диаграмм рекомендуется использовать классические принципы моделирования – декомпозиции и иерархического упорядочения. При моделировании алгоритма вначале строится контекстная диаграмма с деятельностями, которые после детализируются с помощью соответствующих диаграмм декомпозиции.
2. Количество пересечений линий следует минимизировать. При этом считается, что пересекающиеся линии не имеют логической связи друг с другом. Другими словами, потоки данных или управления в местах пересечений не меняют своего направления.
3. Если на диаграмме имеется ветвление / решение на параллельные или альтернативные потоки, то должно указываться и соответствующее соединение / слияние этих потоков.
4. При использовании альтернативных потоков каждый из них должен быть специфицирован с помощью сторожевого условия. Сторожевые условия не должны допускать одновременного срабатывания двух и более переходов.
5. В целях определения зоны ответственности (набора действий) сущностей рекомендуется использовать разделы деятельности.

При разработке диаграммы компонентов следует придерживаться следующих правил:

1. *Перед разработкой диаграмм компонентов необходимо решить, из каких физических частей (файлов) будет состоять программная система. При этом должно быть решено две задачи – распределение классов по файлам исходных кодов и по подсистемам. На этом этапе следует обратить внимание на такую реализацию системы, которая обеспечивала бы возможность повторного использования кода за счет рациональной декомпозиции системы, т. е. минимизировать количество связей между компонентами.*

2. *При спецификации общей структуры исходного кода системы необходимо учитывать специфику языка программирования, с помощью которого реализуются компоненты.*

3. *Для спецификации исполнимого варианта системы необходимо иметь в наличии предварительную топологию системы, т. е. набросок диаграммы развертывания. Для каждого узла в сети может быть построена диаграмма компонентов, определяющая набор файлов, необходимых для работы подсистемы (подсистем) на отдельном рабочем месте.*

4. На диаграмме могут быть представлены отношения зависимости между компонентами и включенными в них классами. Эта информация имеет важное значение для обеспечения согласованности между логическим и физическим представлениями системы. В этом случае зависимость можно показать двумя способами:
- классы показать отдельно от компонента и связать компонент с каждым классом отношением зависимости.
- классы отобразить внутри символа компонента.

5. Рекомендуется минимизировать количество связей между исполняемыми компонентами, а необходимое взаимодействие осуществлять через хранимую в единой базе данных информацию.
6. Для наглядного отображения специфики компонентов можно вместо стандартного символа компонента со строковым стереотипом внутри использовать графические стереотипы.

При разработке диаграммы следует придерживаться следующих правил:

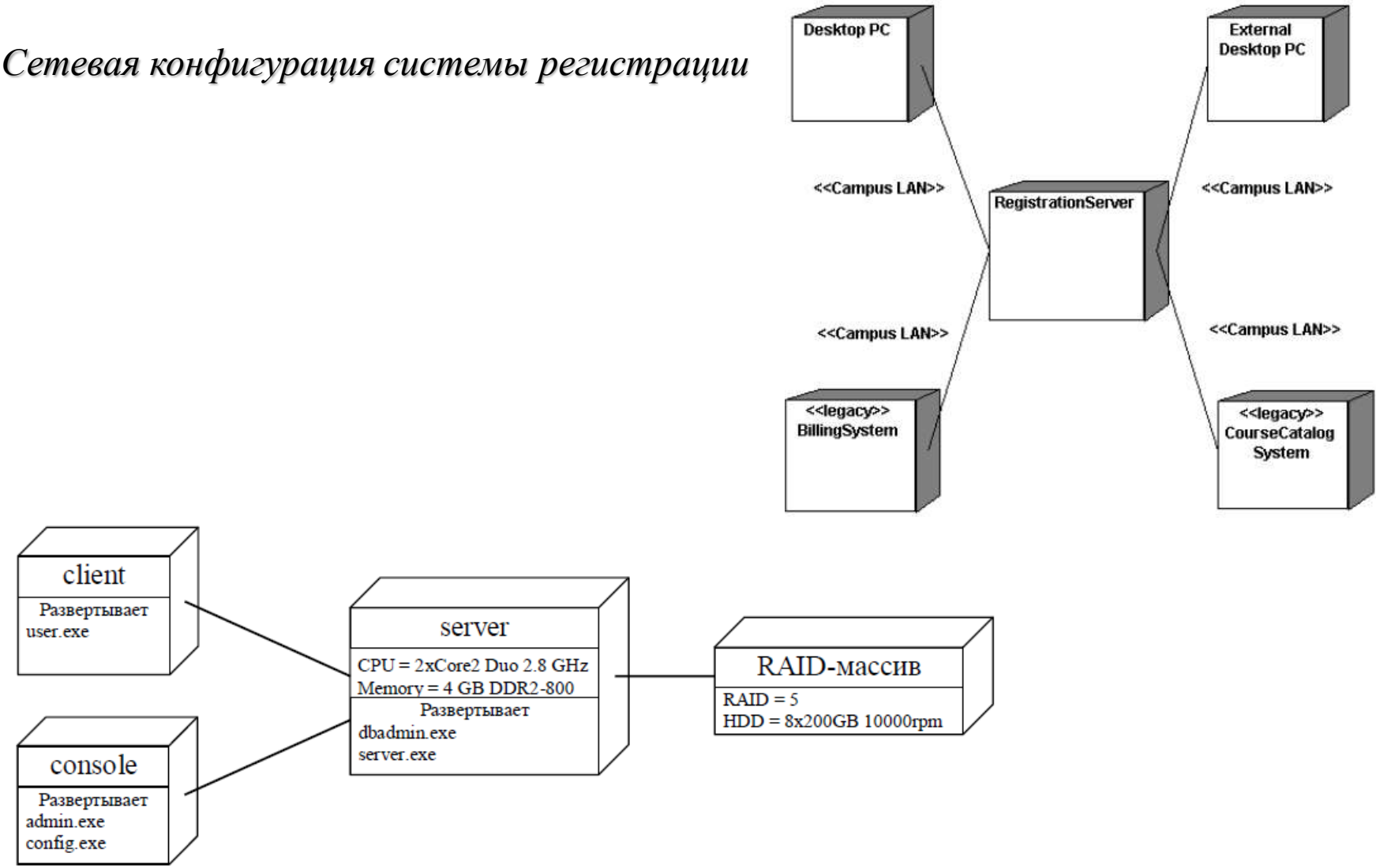
1. Перед разработкой диаграммы развертывания необходимо идентифицировать:
- категории (типы) пользователей. Для каждой категории должны быть определены количество пользователей и требуемые для работы компоненты системы;
- аппаратные, технические и другие типы устройств, необходимые для выполнения системой своих функций;
- виды и требуемую пропускную способность каналов связи.

2. Должны быть рассмотрены варианты прокладки новой или модернизации существующей корпоративной сети организации.

3. В целях наглядного представления распределенной информационной системы на диаграмме рекомендуется отображать компоненты, интерфейсы и связи между ними.

4. Для наглядного отображения специфики узлов вместо стандартного символа узла со строковым стереотипом могут использоваться графические стереотипы.

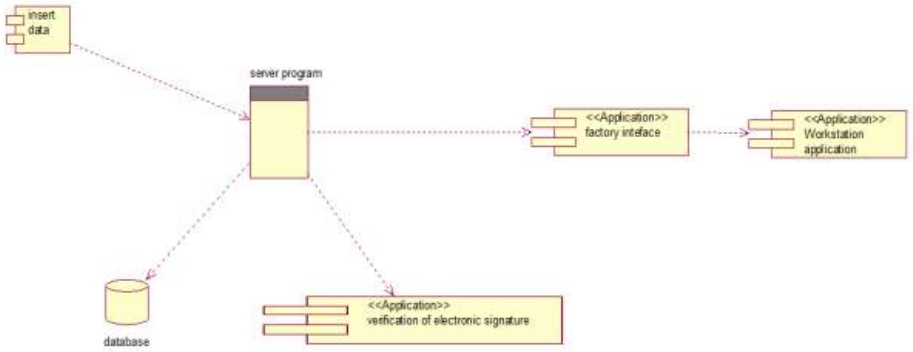
Сетевая конфигурация системы регистрации



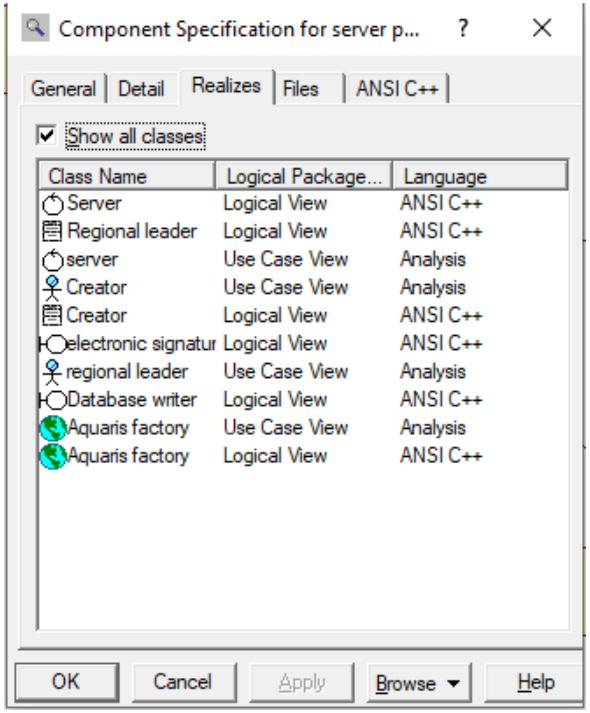
Генерация кода в Rational Rose

1. Создать диаграмму классов ***в logical view.*** *Диаграмма классов должна быть выполнена на английском языке.*

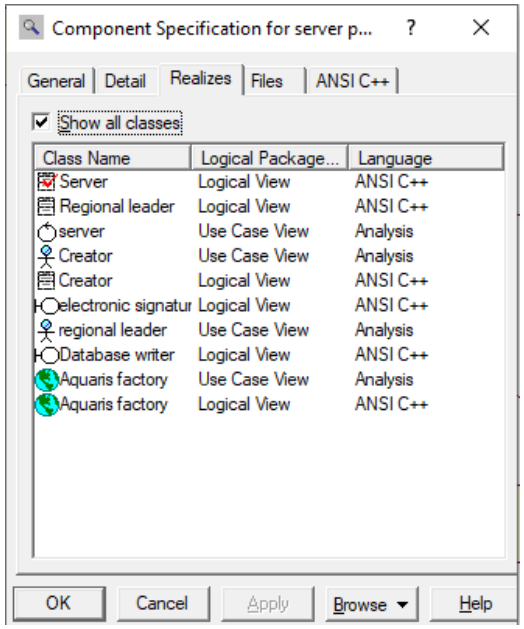
2. Создать диаграмму компонентов. Для этого нажать правой кнопкой мыши на component ***view*** *и выбрать new —> Component Diagram. Данная диаграмма показывает программную реализацию модели, то есть каждый класс должен иметь соответствующий ему элемент на диаграмме компонентов. При необходимости использовать стереотипы Main program (главная программа), Database (база данных), Application (приложение).*



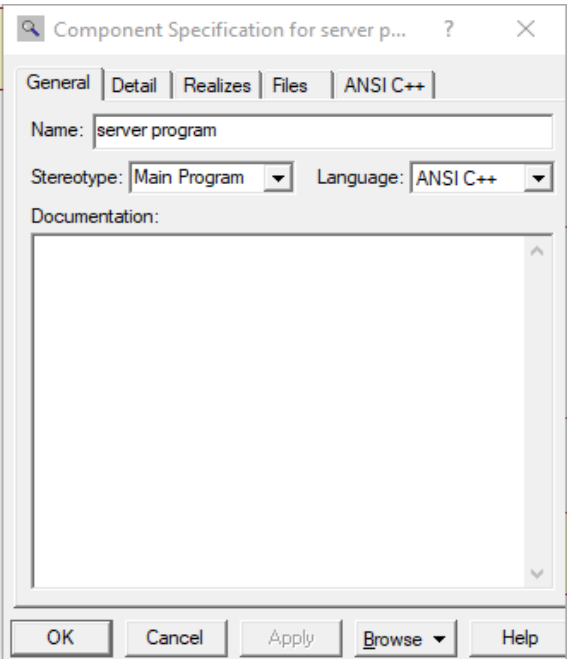
3. Необходимо указать соответствующий каждому компоненту класс. Для этого дважды нажать на компонент, в открывшемся меню перейти на вкладку Realizes.



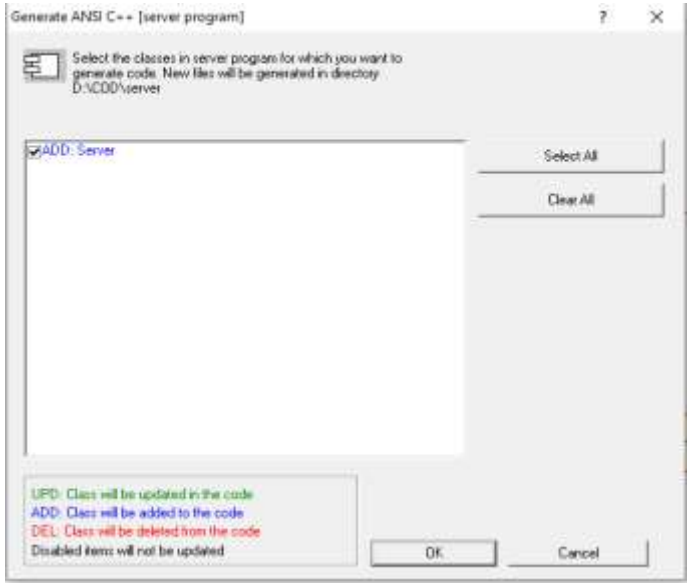
Далее нажать на класс, который реализует компонент правой кнопкой мыши и выбрать Assign в выпадающем меню. После этого на выбранном классе появится красная галочка.



4. После того как каждому компоненту будет поставлен соответствующий класс нужно будет выбрать язык программирования. Для этого нужно опять нажать дважды на компонент, справа в верхнем углу в выпадающем списке выбрать язык программирования (например: ANSI C++). Проделать эту операцию со всеми компонентами.



5. Все предварительные настройки выполнены, можно начинать генерацию кода. Для этого нужно нажать правой кнопкой мыши на компонент, выбрать вкладку ANSI C++ и далее выбрать Generate Code. В появившемся окне нужно указать путь сохранения файлов с будущим кодом. После появится окно генерации:



Проделать данную операцию для каждого компонента.

Для просмотра полученного кода существует два варианта – открыть сгенерированные файлы с помощью среды программирования или нажать на класс в диаграмме классов и сверху на панели меню выбрать ***Tools – ANSI C++ - Browse Header/Browse Body*** *Данные команды отображают код заголовочного файла или файла тела программы соответственно.*

Спасибо за внимание!